REPUBLIQUE TUNISIENNE
* * *
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE
* * *
UNIVERSITE DE SFAX
* * *
FACULTE DES SCIENCES DE SFAX

Compte rendu

Cédule de maintenance de centrales électriques

Formulation et résolution par algorithme génétique
(sélection par loterie, mutation par inversion)

Étudiante : Bouassida Chayma
Filière : Génie logiciel et systèmes d'information – 3e année
Année universitaire : 2026-2027

1. Énoncé et données

On doit planifier la maintenance préventive de 7 unités d'une centrale électrique sur une année découpée en 4 intervalles. Une unité en maintenance est indisponible pendant tout l'intervalle ; la maintenance commence au début d'un intervalle et finit à la fin d'un intervalle. Les unités 1 et 2 ont besoin de 2 intervalles **consécutifs**, les autres d'un seul. Le but est de trouver la cédule qui maximise la puissance en réserve pendant l'année.

Unité	1	2	3	4	5	6	7
Capacité (MW)	20	15	35	40	15	15	10
Intervalles requis	2	2	1	1	1	1	1

Intervalle	1	2	3	4
Demande (MW)	80	90	65	70

La capacité totale est de 20 + 15 + 35 + 40 + 15 + 15 + 10 = **150 MW**.

2. Formulation du problème

Paramètres. $I = \{1, \ldots, 7\}$ les unités, $T = \{1, \ldots, 4\}$ les intervalles, C_i la capacité de l'unité i, d_i sa durée de maintenance (en intervalles), D_t la demande de l'intervalle t et $C_{tot} = 150$ MW.

Variables de décision. $y_{i,s} = 1$ si la maintenance de l'unité i **commence** à l'intervalle s, 0 sinon, avec s possible seulement si $s + d_i - 1 \leq 4$ (la maintenance doit tenir dans l'année). L'unité 1 ou 2 peut donc commencer en s = 1, 2 ou 3 ; les autres unités en s = 1, 2, 3 ou 4.

```
Contrainte 1 (une seule maintenance par unite) :
    pour tout i :   somme s y[i,s] = 1

Puissance perdue pendant l'intervalle t :
    L[t] = somme_i C[i] * ( somme des y[i,s] pour s <= t <= s + d[i] - 1 )

Reserve nette de l'intervalle t :
    R[t] = C tot - L[t] - D[t]

Contrainte 2 (la capacite disponible couvre la demande) :
    pour tout t :   R[t] >= 0

Fonction objectif (maximiser la plus petite reserve) :
    max  z        avec  z <= R[t]  pour tout t
    soit   max  min_t R[t]
```

La contrainte de maintenance consécutive des unités 1 et 2 est intégrée dans les variables $y_{i,s}$: une seule date de début, puis d_i intervalles occupés à la suite. On maximise la **plus petite réserve** sur l'année car c'est l'intervalle le plus tendu qui limite la sécurité du réseau. Le nombre de cédules possibles est $3 \times 3 \times 4^5 =$ **9216** : l'énumération complète est donc possible et sert ici à vérifier l'optimum trouvé par l'algorithme génétique.

3. Encodage du chromosome

Un chromosome contient 7 gènes de 4 bits, un par unité. Le bit k du gène vaut 1 si l'unité est en maintenance à l'intervalle k. Seuls certains motifs représentent une cédule valide :

Unité	Motifs possibles
1 et 2	1100, 0110, 0011

Unité	Motifs possibles
3, 4, 5, 6 et 7	1000, 0100, 0010, 0001

Tous les chromosomes générés, croisés et mutés par nos opérateurs restent valides : un chromosome invalide n'est jamais produit.

4. Fonction de performance (fitness)

Pour un chromosome, on calcule la puissance perdue de chaque intervalle, puis la réserve $R_t = 150 - \text{perdue} - \text{demande}$. La performance est la **plus petite réserve** ; si elle est négative, la performance vaut 0 : $f(\text{chromosome}) = \max(0, \min_t R_t)$. Vérification avec le chromosome d'exemple du cours 0110 0011 0001 1000 0100 0010 1000 :

Intervalle	Unités en maintenance	Perdue (MW)	Demande (MW)	Réserve (MW)
1	4, 7	50	80	20
2	1, 5	35	90	25
3	1, 2, 6	50	65	35
4	2, 3	50	70	30

La performance est min(20, 25, 35, 30) = **20**, comme dans le cours. Le programme refait ce calcul et l'affiche en premier.

5. Opérateurs génétiques

5.1 Sélection par loterie (roulette)

Chaque individu k reçoit une part de la roue proportionnelle à sa performance : probabilité $p_k = f_k$ / somme des f_j. On tire un nombre r uniformément entre 0 et la somme des performances, puis on parcourt la population en cumulant les performances jusqu'à dépasser r. Si tous les individus ont une performance nulle (aucune cédule réalisable), on tire un parent au hasard pour éviter une division par zéro.

5.2 Croisement

Croisement à un point, placé **entre deux gènes** (jamais au milieu d'un bloc de 4 bits), avec une probabilité $p_c = 0{,}8$. Comme le gène de rang i est valide pour l'unité i chez les deux parents, les deux enfants sont toujours valides.

5.3 Mutation par inversion

On choisit au hasard un segment de gènes consécutifs et, dans chaque gène du segment, on **inverse l'ordre des 4 bits**. Cela revient à rendre la maintenance symétrique dans le temps. L'ensemble des motifs valides est stable par inversion : 1100 ↔ 0011, 1000 ↔ 0001, 0100 ↔ 0010, et 0110 reste 0110. Le chromosome obtenu est donc toujours valide.

```
avant  : 0110 0011 0001 1000 0100 0010 1000
segment: genes 2 a 4  ->  0011 0001 1000
apres  : 0110 1100 1000 0001 0100 0010 1000
```

Autres mutations testées. (a) *Inversion de l'ordre des gènes* : on renverse l'ordre des gènes d'un segment ; comme chaque gène a son propre ensemble de motifs, le résultat est souvent invalide (environ 47 % des cas sur des chromosomes aléatoires), et on garde alors le chromosome inchangé. (b) *Remplacement de gène* :

un gène est remplacé par un autre motif valide de la même unité (la mutation de l'exemple du cours, 0001 devient 0100). (c) *Échange de gènes* : deux unités de même durée échangent leur gène. (d) *Aucune mutation*, comme référence.

5.4 Paramètres

Paramètre	Valeur
Taille de la population	50 (le cours conseille 6 à 7 fois le nombre de variables, soit 42 à 49 ; 7 variables)
Générations	50
Probabilité de croisement p_c	0,8
Probabilité de mutation p_m	0,2 par enfant
Élitisme	le meilleur individu est conservé à chaque génération
Population initiale	aléatoire, uniquement des chromosomes valides

6. Résultats

6.1 Une exécution (population 50, mutation par inversion, graine 21)

```
Generation  1 | meilleure = 15 | moyenne =   1.6 | meilleur depuis = gen 1
Generation  2 | meilleure = 15 | moyenne =   8.6 | meilleur depuis = gen 1
Generation  5 | meilleure = 20 | moyenne =  10.2 | meilleur depuis = gen 3
Generation 10 | meilleure = 20 | moyenne =  12.3 | meilleur depuis = gen 3
Generation 15 | meilleure = 25 | moyenne =  16.0 | meilleur depuis = gen 13
Generation 20 | meilleure = 25 | moyenne =  15.4 | meilleur depuis = gen 13
Generation 30 | meilleure = 25 | moyenne =  18.3 | meilleur depuis = gen 13
Generation 50 | meilleure = 25 | moyenne =  20.6 | meilleur depuis = gen 13
```

```
=== Algorithme genetique (selection par loterie, mutation par inversion) ===
Parametres : population 50, 50 generations, pc = 0.8, pm = 0.2, elitisme = 1, graine 21
Generation  1 | meilleure = 15 | moyenne =   1.6 | meilleur depuis = gen 1
Generation  2 | meilleure = 15 | moyenne =   8.6 | meilleur depuis = gen 1
Generation  5 | meilleure = 20 | moyenne =  10.2 | meilleur depuis = gen 3
Generation 10 | meilleure = 20 | moyenne =  12.3 | meilleur depuis = gen 3
Generation 15 | meilleure = 25 | moyenne =  16.0 | meilleur depuis = gen 13
Generation 20 | meilleure = 25 | moyenne =  15.4 | meilleur depuis = gen 13
Generation 30 | meilleure = 25 | moyenne =  18.3 | meilleur depuis = gen 13
Generation 50 | meilleure = 25 | moyenne =  20.6 | meilleur depuis = gen 13
Chromosome : 1100 1100 0001 0010 0010 0001 1000
  Intervalle 1 : unites en maintenance [1, 2, 7] | perdue = 45 MW | demande = 80 | reserve = 25
  Intervalle 2 : unites en maintenance [1, 2] | perdue = 35 MW | demande = 90 | reserve = 25
  Intervalle 3 : unites en maintenance [4, 5] | perdue = 55 MW | demande = 65 | reserve = 30
  Intervalle 4 : unites en maintenance [3, 6] | perdue = 50 MW | demande = 70 | reserve = 30
  Performance (plus petite reserve) = 25
```

La performance passe de 15 à 20 (génération 3) puis à **25** (génération 13). La moyenne de la population monte aussi de 1,6 à environ 20. Cette graine a été choisie parce qu'elle montre bien l'évolution ; la partie 6.3 donne des statistiques sur beaucoup d'exécutions.

Cédule trouvée : chromosome 1100 1100 0001 0010 0010 0001 1000.

Intervalle	Unités en maintenance	Perdue (MW)	Demande (MW)	Réserve (MW)
1	1, 2, 7	45	80	25
2	1, 2	35	90	25
3	4, 5	55	65	30
4	3, 6	50	70	30

6.2 Vérification par énumération

L'énumération des 9216 chromosomes donne une performance maximale de **25 MW**, atteinte par **43** cédules différentes. L'algorithme génétique trouve donc une cédule optimale (25 MW). Exemple de cédule optimale issue de l'énumération : 1100 1100 0010 0001 0010 0001 1000.

6.3 Comparaison des mutations

Chaque variante est exécutée avec les graines 1, 2, 3, … Pour chaque mutation, le tableau donne la meilleure performance moyenne, la pire, le nombre d'exécutions qui atteignent l'optimum (25) et la génération moyenne à laquelle il est trouvé.

Population 50, 50 générations, 30 exécutions :

Mutation	Moyenne	Min	Optimum atteint	Génération moyenne de l'optimum
inversion (bits des blocs)	25,00	25	30 / 30	3,7
inversion (ordre des gènes)	24,67	20	28 / 30	5,6
remplacement de gène	24,33	15	27 / 30	2,9
échange de gènes	24,83	20	29 / 30	3,3
aucune mutation	23,67	15	24 / 30	4,6

Population réduite à 10, 30 générations, 100 exécutions :

Mutation	Moyenne	Min	Optimum atteint	Génération moyenne de l'optimum
inversion (bits des blocs)	19,70	10	34 / 100	7,1
inversion (ordre des gènes)	19,55	10	34 / 100	8,7
remplacement de gène	21,95	10	57 / 100	11,6
échange de gènes	19,95	10	36 / 100	10,4
aucune mutation	14,85	0	7 / 100	1,4

Analyse. Avec 50 individus, toutes les variantes trouvent l'optimum dans presque tous les cas : le problème est petit (9216 cédules, dont 43 optimales). Même sans mutation, une population initiale aléatoire de 50 contient déjà une cédule optimale dans environ 21 % des cas, et l'élitisme la conserve. Sur 30 exécutions, les écarts entre mutations ne sont donc pas significatifs. Avec seulement 10 individus, les différences apparaissent : toute mutation est nettement meilleure que l'absence de mutation (7 exécutions sur 100 atteignent l'optimum sans mutation, contre 34 avec l'inversion). La mutation par inversion préserve la validité mais explore peu : un gène n'a que deux formes accessibles (lui-même et son image miroir). Le remplacement de gène, qui peut atteindre tous les motifs valides, est la meilleure variante dans ce cas (57 sur 100). L'inversion de l'ordre des gènes n'apporte pas de gain par rapport à l'inversion des bits, car près de la moitié de ses mouvements est refusée. Dans la ligne « aucune mutation », la génération moyenne de 1,4 est trompeuse : les rares réussites viennent de la population initiale.

6.4 Capture d'écran de l'exécution

```
=== Verification avec le chromosome d'exemple du cours ===
valide : True | reserves : [20, 25, 35, 30] | performance : 20

=== Algorithme genetique (selection par loterie, mutation par inversion) ===
Parametres : population 50, 50 generations, pc = 0.8, pm = 0.2, elitisme = 1, graine 21
Generation  1 | meilleure = 15 | moyenne =   1.6 | meilleur depuis = gen 1
Generation  2 | meilleure = 15 | moyenne =   8.6 | meilleur depuis = gen 1
Generation  5 | meilleure = 20 | moyenne =  10.2 | meilleur depuis = gen 3
Generation 10 | meilleure = 20 | moyenne =  12.3 | meilleur depuis = gen 3
Generation 15 | meilleure = 25 | moyenne =  16.0 | meilleur depuis = gen 13
Generation 20 | meilleure = 25 | moyenne =  15.4 | meilleur depuis = gen 13
Generation 30 | meilleure = 25 | moyenne =  18.3 | meilleur depuis = gen 13
Generation 50 | meilleure = 25 | moyenne =  20.6 | meilleur depuis = gen 13
Chromosome : 1100 1100 0001 0010 0010 0001 1000
  Intervalle 1 : unites en maintenance [1, 2, 7] | perdue = 45 MW | demande = 80 | reserve = 25
  Intervalle 2 : unites en maintenance [1, 2] | perdue = 35 MW | demande = 90 | reserve = 25
  Intervalle 3 : unites en maintenance [4, 5] | perdue = 55 MW | demande = 65 | reserve = 30
  Intervalle 4 : unites en maintenance [3, 6] | perdue = 50 MW | demande = 70 | reserve = 30
  Performance (plus petite reserve) = 25

=== Optimum exact (enumeration de 9216 chromosomes) ===
Chromosome : 1100 1100 0010 0001 0010 0001 1000
  Intervalle 1 : unites en maintenance [1, 2, 7] | perdue = 45 MW | demande = 80 | reserve = 25
  Intervalle 2 : unites en maintenance [1, 2] | perdue = 35 MW | demande = 90 | reserve = 25
  Intervalle 3 : unites en maintenance [3, 5] | perdue = 50 MW | demande = 65 | reserve = 35
  Intervalle 4 : unites en maintenance [4, 6] | perdue = 55 MW | demande = 70 | reserve = 25
  Performance (plus petite reserve) = 25
  Nombre de chromosomes optimaux : 43 / 9216

=== Comparaison des mutations : population 50, 50 generations, 30 executions ===
mutation                    | moyenne | min | optimum atteint | generation moyenne de l'optimum
inversion (bits des blocs)  |   25.00 |  25 |  30 / 30        | 3.7
inversion (ordre des genes) |   24.67 |  20 |  28 / 30        | 5.6
remplacement de gene        |   24.33 |  15 |  27 / 30        | 2.9
echange de genes            |   24.83 |  20 |  29 / 30        | 3.3
aucune mutation             |   23.67 |  15 |  24 / 30        | 4.6

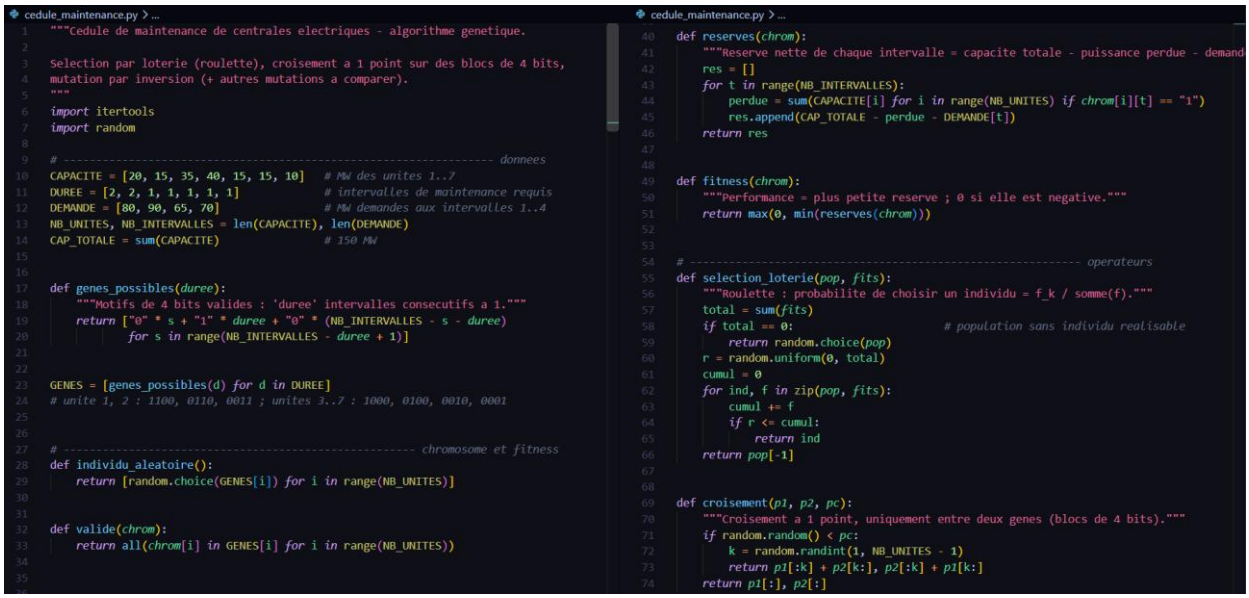
=== Comparaison des mutations : population 10, 30 generations, 100 executions ===
mutation                    | moyenne | min | optimum atteint | generation moyenne de l'optimum
inversion (bits des blocs)  |   19.70 |  10 |  34 / 100       | 7.1
inversion (ordre des genes) |   19.55 |  10 |  34 / 100       | 8.7
remplacement de gene        |   21.95 |  10 |  57 / 100       | 11.6
echange de genes            |   19.95 |  10 |  36 / 100       | 10.4
aucune mutation             |   14.85 |   0 |   7 / 100       | 1.4
```

Figure 1 - Exécution de cedule_maintenance.py (VS Code)

7. Conclusion

Le problème de cédule de maintenance se formule comme une maximisation de la plus petite réserve de puissance, sous des contraintes de durée de maintenance et de couverture de la demande. L'encodage par gènes de 4 bits rend toutes les cédules générées valides, et l'algorithme génétique avec sélection par loterie, croisement entre gènes et mutation par inversion trouve l'optimum de 25 MW, confirmé par l'énumération. La comparaison montre que la mutation est utile quand la population est petite, et que le remplacement de gène explore mieux l'espace que l'inversion seule, qui a l'avantage de toujours produire une cédule valide.

Annexe : captures d'écran du code



```python
"""Cedule de maintenance de centrales electriques - algorithme genetique.

Selection par loterie (roulette), croisement a 1 point sur des blocs de 4 bits,
mutation par inversion (+ autres mutations a comparer).
"""
import itertools
import random

# ---------------------------------------------------------------- donnees
CAPACITE = [20, 15, 35, 40, 15, 15, 10]   # MW des unites 1..7
DUREE = [2, 2, 1, 1, 1, 1, 1]             # intervalles de maintenance requis
DEMANDE = [80, 90, 65, 70]                # MW demandes aux intervalles 1..4
NB_UNITES, NB_INTERVALLES = len(CAPACITE), len(DEMANDE)
CAP_TOTALE = sum(CAPACITE)                # 150 MW


def genes_possibles(duree):
    """Motifs de 4 bits valides : 'duree' intervalles consecutifs a 1."""
    return ["0" * s + "1" * duree + "0" * (NB_INTERVALLES - s - duree)
            for s in range(NB_INTERVALLES - duree + 1)]


GENES = [genes_possibles(d) for d in DUREE]
# unite 1, 2 : 1100, 0110, 0011 ; unites 3..7 : 1000, 0100, 0010, 0001


# ------------------------------------------------------ chromosome et fitness
def individu_aleatoire():
    return [random.choice(GENES[i]) for i in range(NB_UNITES)]


def valide(chrom):
    return all(chrom[i] in GENES[i] for i in range(NB_UNITES))
```

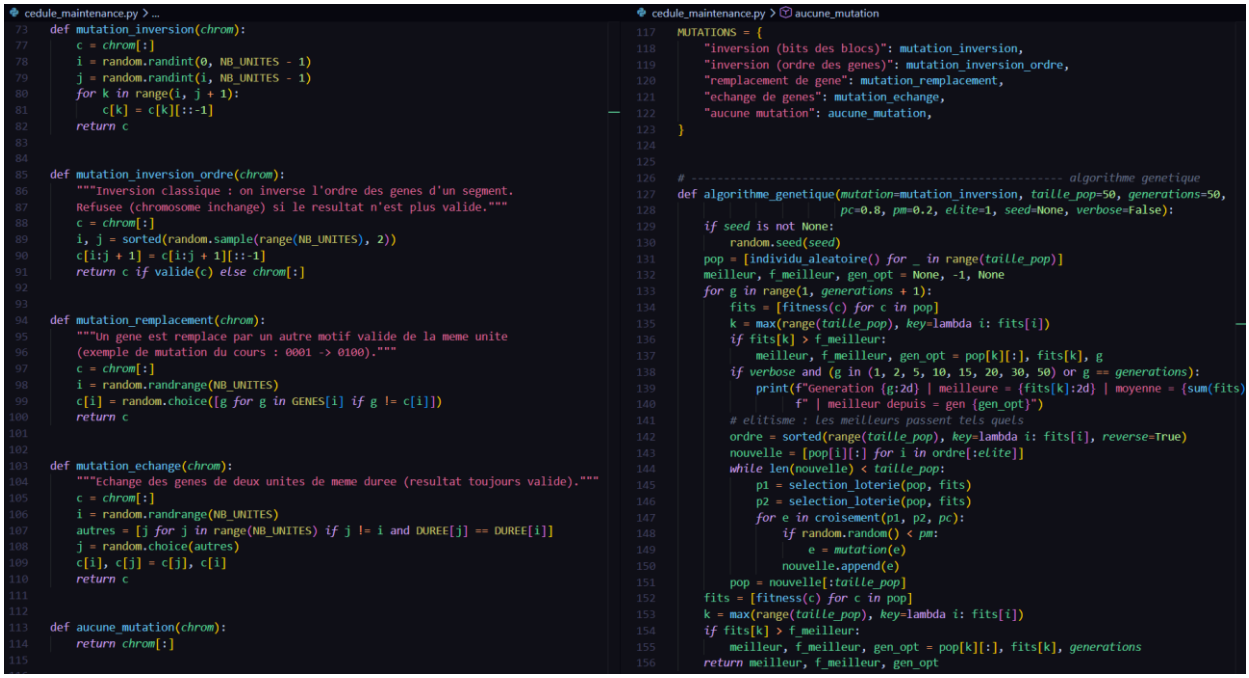
```python
def reserves(chrom):
    """Reserve nette de chaque intervalle = capacite totale - puissance perdue - demand
    res = []
    for t in range(NB_INTERVALLES):
        perdue = sum(CAPACITE[i] for i in range(NB_UNITES) if chrom[i][t] == "1")
        res.append(CAP_TOTALE - perdue - DEMANDE[t])
    return res


def fitness(chrom):
    """Performance = plus petite reserve ; 0 si elle est negative."""
    return max(0, min(reserves(chrom)))


# ------------------------------------------------------------ operateurs
def selection_loterie(pop, fits):
    """Roulette : probabilite de choisir un individu = f_k / somme(f)."""
    total = sum(fits)
    if total == 0:                        # population sans individu realisable
        return random.choice(pop)
    r = random.uniform(0, total)
    cumul = 0
    for ind, f in zip(pop, fits):
        cumul += f
        if r <= cumul:
            return ind
    return pop[-1]


def croisement(p1, p2, pc):
    """Croisement a 1 point, uniquement entre deux genes (blocs de 4 bits)."""
    if random.random() < pc:
        k = random.randint(1, NB_UNITES - 1)
        return p1[:k] + p2[k:], p2[:k] + p1[k:]
    return p1[:], p2[:]
```

Figure 2 - cedule_maintenance.py : données, gènes possibles, fonction de performance, sélection, croisement



```python
def mutation_inversion(chrom):
    c = chrom[:]
    i = random.randint(0, NB_UNITES - 1)
    j = random.randint(i, NB_UNITES - 1)
    for k in range(i, j + 1):
        c[k] = c[k][::-1]
    return c


def mutation_inversion_ordre(chrom):
    """Inversion classique : on inverse l'ordre des genes d'un segment.
    Refusee (chromosome inchange) si le resultat n'est plus valide."""
    c = chrom[:]
    i, j = sorted(random.sample(range(NB_UNITES), 2))
    c[i:j + 1] = c[i:j + 1][::-1]
    return c if valide(c) else chrom[:]


def mutation_remplacement(chrom):
    """Un gene est remplace par un autre motif valide de la meme unite
    (exemple de mutation du cours : 0001 -> 0100)."""
    c = chrom[:]
    i = random.randrange(NB_UNITES)
    c[i] = random.choice([g for g in GENES[i] if g != c[i]])
    return c

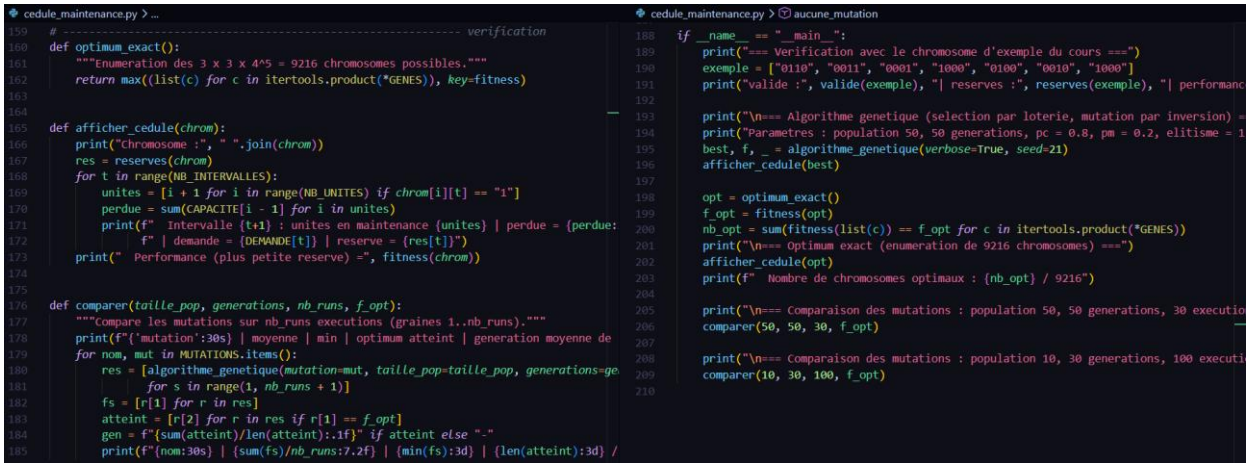

def mutation_echange(chrom):
    """Echange des genes de deux unites de meme duree (resultat toujours valide)."""
    c = chrom[:]
    i = random.randrange(NB_UNITES)
    autres = [j for j in range(NB_UNITES) if j != i and DUREE[j] == DUREE[i]]
    j = random.choice(autres)
    c[i], c[j] = c[j], c[i]
    return c


def aucune_mutation(chrom):
    return chrom[:]
```

```python
MUTATIONS = {
    "inversion (bits des blocs)": mutation_inversion,
    "inversion (ordre des genes)": mutation_inversion_ordre,
    "remplacement de gene": mutation_remplacement,
    "echange de genes": mutation_echange,
    "aucune mutation": aucune_mutation,
}


# ------------------------------------------------------ algorithme genetique
def algorithme_genetique(mutation=mutation_inversion, taille_pop=50, generations=50,
                         pc=0.8, pm=0.2, elite=1, seed=None, verbose=False):
    if seed is not None:
        random.seed(seed)
    pop = [individu_aleatoire() for _ in range(taille_pop)]
    meilleur, f_meilleur, gen_opt = None, -1, None
    for g in range(1, generations + 1):
        fits = [fitness(c) for c in pop]
        k = max(range(taille_pop), key=lambda i: fits[i])
        if fits[k] > f_meilleur:
            meilleur, f_meilleur, gen_opt = pop[k][:], fits[k], g
        if verbose and (g in (1, 2, 5, 10, 15, 20, 30, 50) or g == generations):
            print(f"Generation {g:2d} | meilleure = {fits[k]:2d} | moyenne = {sum(fits)
                  f" | meilleur depuis = gen {gen_opt}")
        # elitisme : les meilleurs passent tels quels
        ordre = sorted(range(taille_pop), key=lambda i: fits[i], reverse=True)
        nouvelle = [pop[i][:] for i in ordre[:elite]]
        while len(nouvelle) < taille_pop:
            p1 = selection_loterie(pop, fits)
            p2 = selection_loterie(pop, fits)
            for e in croisement(p1, p2, pc):
                if random.random() < pm:
                    e = mutation(e)
                nouvelle.append(e)
        pop = nouvelle[:taille_pop]
    fits = [fitness(c) for c in pop]
    k = max(range(taille_pop), key=lambda i: fits[i])
    if fits[k] > f_meilleur:
        meilleur, f_meilleur, gen_opt = pop[k][:], fits[k], generations
    return meilleur, f_meilleur, gen_opt
```

Figure 3 - cedule_maintenance.py : mutations et algorithme génétique



```python
# ------------------------------------------------------------ verification
def optimum_exact():
    """Enumeration des 3 x 3 x 4^5 = 9216 chromosomes possibles."""
    return max((list(c) for c in itertools.product(*GENES)), key=fitness)


def afficher_cedule(chrom):
    print("Chromosome :", " ".join(chrom))
    res = reserves(chrom)
    for t in range(NB_INTERVALLES):
        unites = [i + 1 for i in range(NB_UNITES) if chrom[i][t] == "1"]
        perdue = sum(CAPACITE[i - 1] for i in unites)
        print(f"  Intervalle {t+1} : unites en maintenance {unites} | perdue = {perdue:
              f" | demande = {DEMANDE[t]} | reserve = {res[t]}")
    print("  Performance (plus petite reserve) =", fitness(chrom))


def comparer(taille_pop, generations, nb_runs, f_opt):
    """Compare les mutations sur nb_runs executions (graines 1..nb_runs)."""
    print(f"{'mutation':30s} | moyenne | min | optimum atteint | generation moyenne de
    for nom, mut in MUTATIONS.items():
        res = [algorithme_genetique(mutation=mut, taille_pop=taille_pop, generations=ge
               for s in range(1, nb_runs + 1)]
        fs = [r[1] for r in res]
        atteint = [r[2] for r in res if r[1] == f_opt]
        gen = f"{sum(atteint)/len(atteint):.1f}" if atteint else "-"
        print(f"{nom:30s} | {sum(fs)/nb_runs:7.2f} | {min(fs):3d} | {len(atteint):3d} /
```

```python
if __name__ == "__main__":
    print("=== Verification avec le chromosome d'exemple du cours ===")
    exemple = ["0110", "0011", "0001", "1000", "0100", "0010", "1000"]
    print("valide :", valide(exemple), "| reserves :", reserves(exemple), "| performanc

    print("\n=== Algorithme genetique (selection par loterie, mutation par inversion) =
    print("Parametres : population 50, 50 generations, pc = 0.8, pm = 0.2, elitisme = 1
    best, f, _ = algorithme_genetique(verbose=True, seed=21)
    afficher_cedule(best)

    opt = optimum_exact()
    f_opt = fitness(opt)
    nb_opt = sum(fitness(list(c)) == f_opt for c in itertools.product(*GENES))
    print("\n=== Optimum exact (enumeration de 9216 chromosomes) ===")
    afficher_cedule(opt)
    print(f"  Nombre de chromosomes optimaux : {nb_opt} / 9216")

    print("\n=== Comparaison des mutations : population 50, 50 generations, 30 executio
    comparer(50, 50, 30, f_opt)

    print("\n=== Comparaison des mutations : population 10, 30 generations, 100 executi
    comparer(10, 30, 100, f_opt)
```

Figure 4 - cedule_maintenance.py : vérification, comparaison et programme principal